

Checker

Checker

Checker, 即 [Special Judge](#), 用于检验答案是否合法。使用 Testlib 可以让我们免去检验许多东西, 使编写简单许多。

Checker 从命令行参数读取到输入文件名、选手输出文件名、标准输出文件名, 并确定选手输出是否正确, 并返回一个预定义的结果:

请在阅读下文前先阅读 [通用](#)。

简单的例子

▼ Note

给定两个整数 (), 输出它们的和。

这题显然不需要 checker 对吧, 但是如果一定要的话也可以写一个:

```
1 #include "testlib.h"
2
3 int main(int argc, char* argv[]) {
4     registerTestlibCmd(argc, argv);
5
6     int pans = ouf.readInt(-2000, 2000, "sum of numbers");
7
8     // 假定标准输出是正确的, 不检查其范围
9     // 之后我们会看到这并不合理
10    int jans = ans.readInt();
11
12    if (pans == jans)
13        quitf(_ok, "The sum is correct.");
14    else
15        quitf(_wa, "The sum is wrong: expected = %d, found = %d", jans, pans);
16 }
```

编写 readAns 函数

假设你有一道题输入输出均有很多数, 如: 给定一张 DAG, 求 到 的最长路并输出路径 (可能有多条, 输出任一)。

下面是一个 不好 的 checker 的例子。

不好的实现

```
1 // clang-format off
2
3 #include "testlib.h"
4 #include <LMap>
5 #include <LVector>
6 using namespace std;
7
8 map<pair<int, int>, int> edges;
9
10 int main(int argc, char* argv[]) {
11     registerTestlibCmd(argc, argv);
```

```

12 int n = inf.readInt(); // 不需要 readSpace() 或 readEoln()
13 int m = inf.readInt(); // 因为不需要在 checker 中检查标准输入合法性
14 // (有 validator)
15 for (int i = 0; i < m; i++) {
16     int a = inf.readInt();
17     int b = inf.readInt();
18     int w = inf.readInt();
19     edges[make_pair(a, b)] = edges[make_pair(b, a)] = w;
20 }
21 int s = inf.readInt();
22 int t = inf.readInt();
23
24 // 读入标准输出
25 int jvalue = 0;
26 vector<int> jpath;
27 int jlen = ans.readInt();
28 for (int i = 0; i < jlen; i++) {
29     jpath.push_back(ans.readInt());
30 }
31 for (int i = 0; i < jlen - 1; i++) {
32     jvalue += edges[make_pair(jpath[i], jpath[i + 1])];
33 }
34
35 // 读入选手输出
36 int pvalue = 0;
37 vector<int> ppath;
38 vector<bool> used(n);
39 int plen = ouf.readInt(2, n, "number of vertices"); // 至少包含 s 和 t 两个点
40 for (int i = 0; i < plen; i++) {
41     int v = ouf.readInt(1, n, format("path[%d]", i + 1).c_str());
42     if (used[v - 1]) // 检查每条边是否只用到一次
43         quitf(_wa, "vertex %d was used twice", v);
44     used[v - 1] = true;
45     ppath.push_back(v);
46 }
47 // 检查起点终点合法性
48 if (ppath.front() != s)
49     quitf(_wa, "path doesn't start in s: expected s = %d, found %d", s,
50         ppath.front());
51 if (ppath.back() != t)
52     quitf(_wa, "path doesn't finish in t: expected t = %d, found %d", t,
53         ppath.back());
54 // 检查相邻点间是否有边
55 for (int i = 0; i < plen - 1; i++) {
56     if (edges.find(make_pair(ppath[i], ppath[i + 1])) == edges.end())
57         quitf(_wa, "there is no edge (%d, %d) in the graph", ppath[i],
58             ppath[i + 1]);
59     pvalue += edges[make_pair(ppath[i], ppath[i + 1])];
60 }
61
62 if (jvalue != pvalue)
63     quitf(_wa, "jury has answer %d, participant has answer %d", jvalue, pvalue);
64 else
65     quitf(_ok, "answer = %d", pvalue);
66 }

```

这个 checker 主要有两个问题：

1. 它确信标准输出是正确的。如果选手输出比标准输出更优，它会被判成 WA，这不太妙。同时，如果标准输出不合法，也会产生 WA。对于这两种情况，正确的操作都是返回 Fail 状态。
2. 读入标准输出和选手输出的代码是重复的。在这道题中写两遍读入问题不大，只需要一个 for 循环；但是如果有一道题输出很复杂，就会导致你的 checker 结构混乱。重复代码会大大降低可维护性，让你在 debug 或修改格式时变得困难。

读入标准输出和选手输出的方式实际上是完全相同的，这就是我们通常编写一个用流作为参数的读入函数的原因。

好的实现

```
1 // clang-format off
2
3 #include "testlib.h"
4 #include <LMap>
5 #include <LVector>
6 using namespace std;
7
8 map<pair<int, int>, int> edges;
9 int n, m, s, t;
10
11 // 这个函数接受一个流，从其中读入
12 // 检查路径的合法性并返回路径长度
13 // 当 stream 为 ans 时，所有 stream.quitf(_wa, ...)
14 // 和失败的 readXxx() 均会返回 _fail 而非 _wa
15 // 也就是说，如果输出非法，对于选手输出流它将返回 _wa，
16 // 对于标准输出流它将返回 _fail
17 int readAns(InStream& stream) {
18     // 读入输出
19     int value = 0;
20     vector<int> path;
21     vector<bool> used(n);
22     int len = stream.readInt(2, n, "number of vertices");
23     for (int i = 0; i < len; i++) {
24         int v = stream.readInt(1, n, format("path[%d]", i + 1).c_str());
25         if (used[v - 1]) {
26             stream.quitf(_wa, "vertex %d was used twice", v);
27         }
28         used[v - 1] = true;
29         path.push_back(v);
30     }
31     if (path.front() != s)
32         stream.quitf(_wa, "path doesn't start in s: expected s = %d, found %d", s,
33             path.front());
34     if (path.back() != t)
35         stream.quitf(_wa, "path doesn't finish in t: expected t = %d, found %d", t,
36             path.back());
37     for (int i = 0; i < len - 1; i++) {
38         if (edges.find(make_pair(path[i], path[i + 1])) == edges.end())
39             stream.quitf(_wa, "there is no edge (%d, %d) in the graph", path[i],
40                 path[i + 1]);
41         value += edges[make_pair(path[i], path[i + 1])];
42     }
43     return value;
44 }
45
46 int main(int argc, char* argv[]) {
47     registerTestlibCmd(argc, argv);
48     n = inf.readInt();
49     m = inf.readInt();
50     for (int i = 0; i < m; i++) {
51         int a = inf.readInt();
52         int b = inf.readInt();
53         int w = inf.readInt();
54         edges[make_pair(a, b)] = edges[make_pair(b, a)] = w;
55     }
56     int s = inf.readInt();
57     int t = inf.readInt();
58
59     int jans = readAns(ans);
60     int pans = readAns(ouf);
```

```

61  if (jans > pans)
62      quitf(_wa, "jury has the better answer: jans = %d, pans = %d\n", jans,
63              pans);
64  else if (jans == pans)
65      quitf(_ok, "answer = %d\n", pans);
66  else // (jans < pans)
67      quitf(_fail, ":( participant has the better answer: jans = %d, pans = %d\n",
68              jans, pans);
69 }

```

注意到这种写法我们同时也检查了标准输出是否合法，这样写 checker 让程序更短，且易于理解和 debug。此种写法也适用于输出 YES（并输出方案什么的），或 NO 的题目。

▼ Note

对于某些限制的检查可以用 `InStream::ensure/ensuref()` 函数更简洁地实现。如上例第 23 至 25 行也可以等价地写成如下形式：

```
1 stream.ensuref(!used[v - 1], "vertex %d was used twice", v);
```

▼ Warning

请在 `readAns` 中避免调用 全局 函数 `::ensure/ensuref()`，这会导致在某些应判为 WA 的选手输出下返回 `_fail`，产生错误。

建议与常见错误

- 编写 `readAns` 函数，它真的可以让你的 checker 变得很棒。
- 读入选手输出时永远限定好范围，如果某些变量忘记了限定且被用于某些参数，你的 checker 可能会判定错误或 RE 等。

◦ 反面教材

```

1 // ....
2 int k = ouf.readInt();
3 vector<int> lst;
4 for (int i = 0; i < k; i++) // k = 0 和 k = -5 在这里作用相同（不会进入循环体）
5     lst.push_back(ouf.readInt());
6 // 但是我们并不想接受一个长度为 -5 的 list，不是吗？
7 // ....
8 int pos = ouf.readInt();
9 int x = A[pos];
10 // 可能会有人输出 -42, 2147483456 或其他一些非法数字导致 checker RE

```

◦ 正面教材

```

1 // ....
2 int k = ouf.readInt(0, n); // 长度不合法会立刻判 WA 而不会继续 check 导致 RE
3 vector<int> lst;
4 for (int i = 0; i < k; i++) lst.push_back(ouf.readInt());
5 // ....
6 int pos = ouf.readInt(0, (int)A.size() - 1); // 防止 out of range
7 int x = A[pos];
8 // ....

```

- 使用项别名。
- 和 `validator` 不同，`checker` 不用特意检查非空字符。例如对于一个按顺序比较整数的 checker，我们只需判断选手输出的整数和答案整数是否对应相等，而选手是每行输出一个整数，还是在一行中输出所有整数等格式问题，

我们的 checker 不必关心。

使用方法

通常我们不需要本地运行它，评测工具/OJ 会帮我们做好这一切。但是如果需要的话，以以下格式在命令行运行：

```
1 ./checker &LTinput-file> &LToutput-file> &LTanswer-file> [&LTreport-file> [&LTappes>]]
```

一些预设的 checker

很多时候我们的 checker 完成的工作很简单（如判断输出的整数是否正确，输出的浮点数是否满足精度要求），[Testlib](#) 已经为我们给出了这些 checker 的实现，我们可以直接使用。

一些常用的 checker 有：

- ncmp：按顺序比较 64 位整数。
- rcmp4：按顺序比较浮点数，最大可接受误差（绝对误差或相对误差）不超过（还有 rcmp6，rcmp9 等对精度要求不同的 checker，用法和 rcmp4 类似）。
- wcmp：按顺序比较字符串（不带空格，换行符等非空字符）。
- yesno：比较 YES 和 NO，大小写不敏感。

本文主要翻译自 [Checkers with testlib.h - Codeforces](#)。testlib.h 的 GitHub 存储库为 [MikeMirzayanov/testlib](#)。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[ayuusweetfish](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [NachtgeistW](#), [ouuan](#), [ouuan](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用